The schema ownership goes to the database, and the data access layer must assist the Domain Model to communicate with the underlying data.

7.4 Entity state transitions

JPA shifts the developer mindset from SQL statements to entity state transitions. An entity can be in one of the following states:

Table 7.1: JPA entity states

State	Description
New (Transient)	A newly created entity which is not mapped to any database row is considered to be in the New or Transient state. Once it becomes managed, the Persistence Context issues an insert statement at flush time.
Managed (Persistent)	A Persistent entity is associated with a database row, and it is being managed by the currently running Persistence Context. State changes are detected by the *dirty checking* mechanism and propagated to the database as update statements at flush time.
Detached	Once the currently running Persistence Context is closed, all the previously managed entities become detached. Successive changes are no longer tracked, and no automatic database synchronization is going to happen.
Removed	A removed entity is only scheduled for deletion, and the actual database delete statement is executed during Persistence Context flushing.

The Persistence Context captures entity state changes, and, during flushing, it translates them into SQL statements. The JPA `EntityManager`[4] and the Hibernate `Session`[5] (which includes additional methods for moving an entity from one state to the other) interfaces are gateways towards the underlying Persistence Context, and they define all the entity state transition operations.

[4] http://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html#persist-java.lang.Object-
[5] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.html

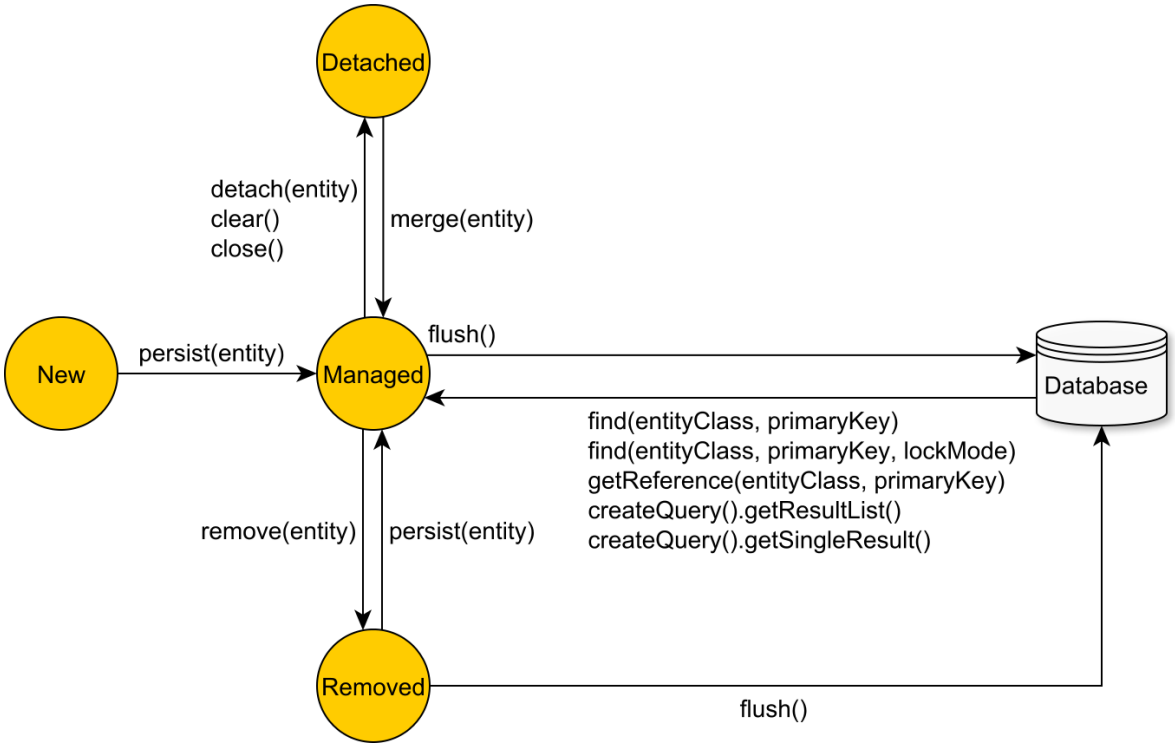

Figure 7.4: JPA entity state transitions

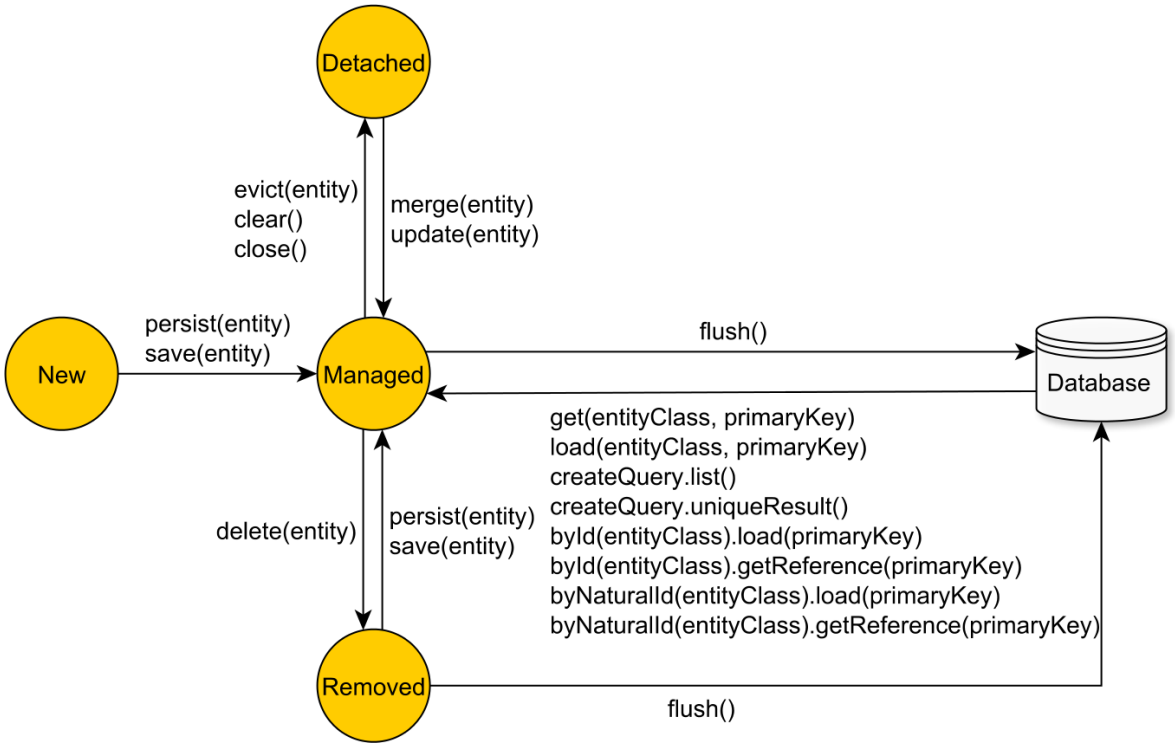

Figure 7.5: Hibernate entity state transitions

7.5 Write-based optimizations

SQL injection prevention

By managing the SQL statement generation, the JPA tool can assist in minimizing the risk of SQL injection attacks. The less the chance of manipulating SQL `String` statements, the safer the application can get. The risk is not completely eliminated because the application developer can still recur to concatenating SQL or JPQL fragments, so rigor is advised.

Hibernate uses `PreparedStatement(s)` exclusively, so not only it protects against SQL injection, but the data access layer can better take advantage of server-side and client-side statement caching as well.

Auto-generated DML statements

The enterprise system database schema evolves with time, and the data access layer must mirror all these modifications as well.

Because the JPA provider auto-generates insert and update statements, the data access layer can easily accommodate database table structure modifications. By updating the entity model schema, Hibernate can automatically adjust the modifying statements accordingly.

This applies to changing database column types as well. If the database schema needs to migrate a postal code from an `INT` database type to a `VARCHAR(6)`, the data access layer will need only to change the associated Domain Model attribute type from an `Integer` to a `String`, and all statements are going to be automatically updated. Hibernate defines a highly customizable JDBC-to-database type mapping system, and the application developer can override a default type association, or even add support for new database types (that are not currently supported by Hibernate).

The entity fetching process is automatically managed by the JPA implementation, which auto-generates the select statements of the associated database tables. This way, JPA can free the application developer from maintaining entity selection queries as well.

Hibernate allows customizing all the CRUD statements, in which case the application developer is responsible for maintaining the associated DML statements.

Although it takes care of the entity selection process, most enterprise systems need to take advantage of the underlying database querying capabilities. For this reason, whenever the database schema changes, all the native SQL queries need to be updated manually (according to their associated business logic requirements).

Write-behind cache

The Persistence Context acts as a transactional write-behind cache, deferring entity state flushing up until the last possible moment.

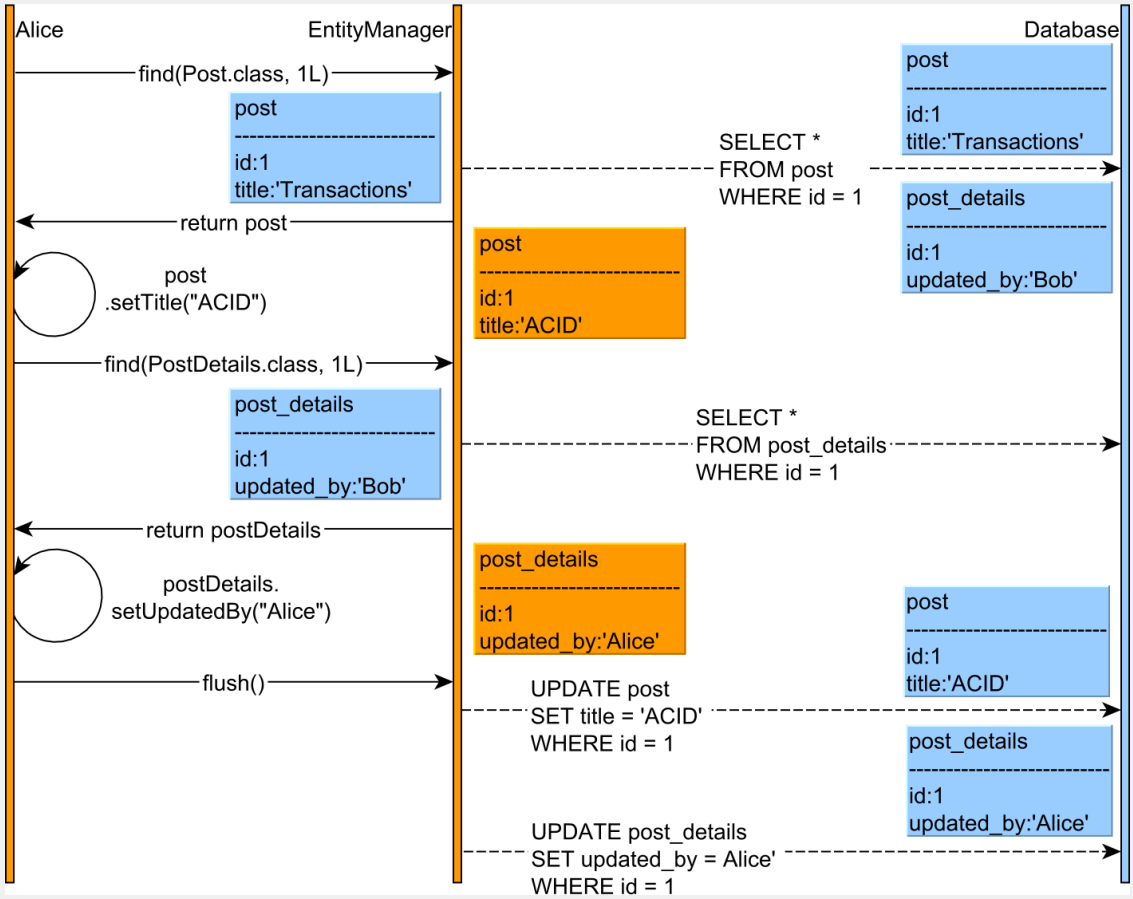

Figure 7.6: Persistence Context

Because every modifying DML statement requires locking (to prevent dirty writes), the write behind cache can reduce the database lock acquisition interval, therefore increasing concurrency.

However, caches introduce consistency challenges, and the Persistence Context requires a flush prior to executing any JPQL or native SQL query, as otherwise it might break the read-your-own-write consistency guarantee.

As detailed in the following chapters, Hibernate does not automatically flush pending changes when a native query is about to be executed, and the application developer must explicitly instruct what database tables are needed to be synchronized.

Transparent statement batching

Since all changes are being flushed at once, Hibernate may benefit from batching JDBC statements. Batch updates can be enabled transparently, even after the data access logic has been implemented. Most often, performance tuning is postponed until the system is already running in production, and switching to batching statements should not require a considerable development effort.

With just one configuration, Hibernate can execute all prepared statements in batches.

Application-level concurrency control

As previously explained, no database isolation level can protect against losing updates when executing a multi-request long conversation. JPA supports both optimistic and pessimistic locking.

The JPA optimistic locking mechanism allows preventing lost updates because it imposes a *happens-before* event ordering. However, in multi-request conversations, optimistic locking requires maintaining old entity snapshots, and JPA makes it possible through *Extended Persistence Contexts* or detached entities.

A Java EE application server can preserve a given Persistence Context across several web requests, therefore providing application-level repeatable reads. However, this strategy is not free since the application developer must make sure the Persistence Context is not bloated with too many entities, which, apart from consuming memory, it can also affect the performance of the Hibernate default dirty checking mechanism.

Even when not using Java EE, the same goal can be achieved using detached entities, which provide fine-grained control over the amount of data needed to be preserved from one web request to the other. JPA allows merging detached entities, which rebecome managed and automatically synchronized with the underlying database system.

JPA also supports a pessimistic locking query abstraction, which comes in handy when using lower-level transaction isolation modes.

Hibernate has a native pessimistic locking API, which brings support for timing out lock acquisition requests or skipping already acquired locks.